Interactive Applications and Cloud Deployment

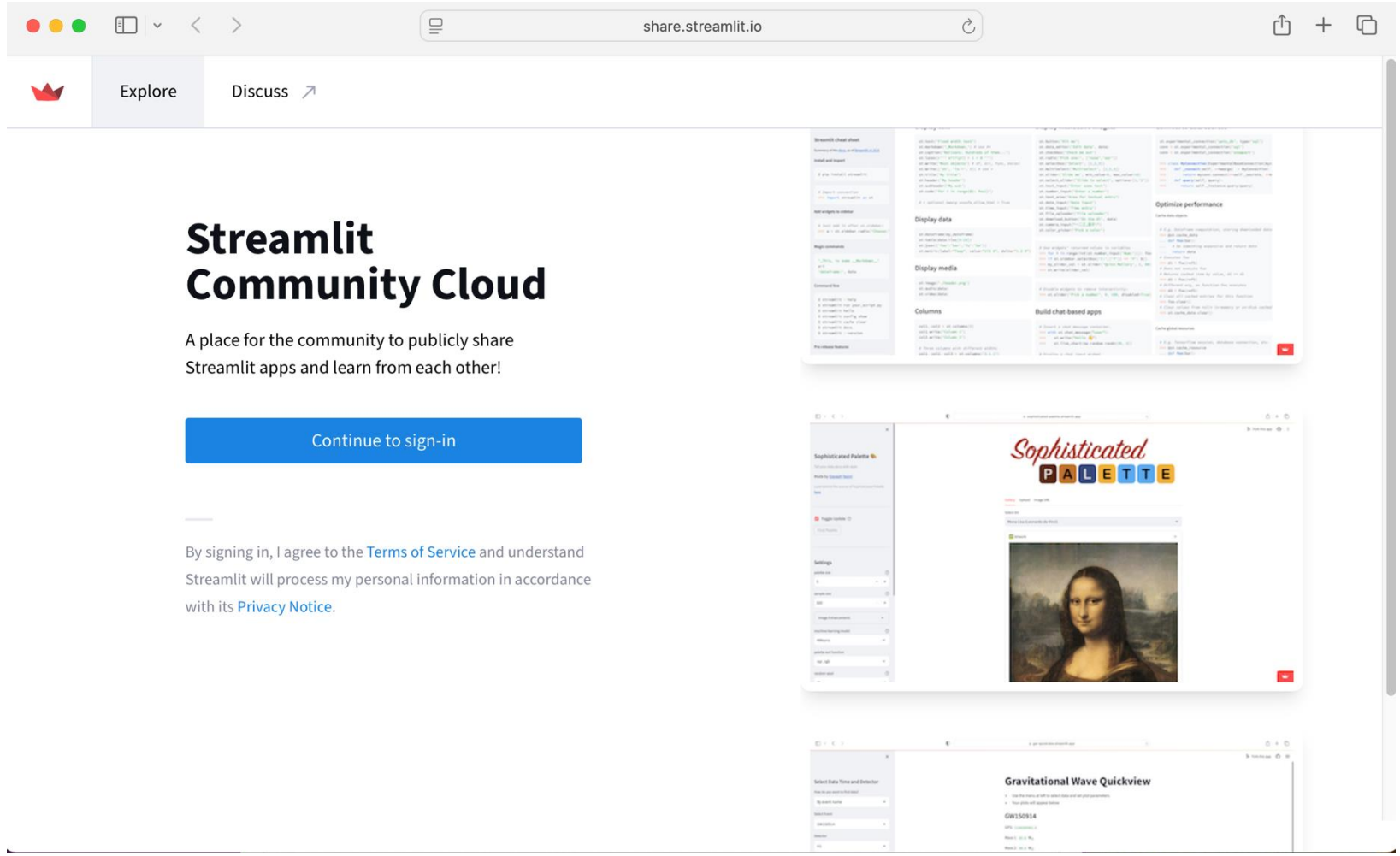
Streamlit

A faster way to build and share data apps

Turn your data scripts into shareable web apps in minutes.
All in pure Python. No front-end experience required.

Key Advantages:

- Pure Python - leverage your existing skills
- No web development required - no HTML, CSS, or JavaScript
- Rapid development - prototype in minutes, not days
- Automatic reactivity - UI updates automatically
- Rich widget library - inputs, charts, media, and more
- Easy deployment - multiple cloud options
- Active community - extensive examples and support



Also, has free deployment but also need to create an account. Lots of examples.

https://share.streamlit.io/explore?sort=most+viewed&category=favorites

Streamlit vs Traditional Web Development

Streamlit Application:

Single Python script - no separation needed
Automatic state management and reactivity
Simple deployment with one command
Only Python knowledge required
Rapid prototyping and iteration
Minimal boilerplate code

Traditional Web Application:

Separate frontend (HTML/CSS/JS) and backend (Python/Node/etc)
Manual state management and synchronization
Complex build and deployment pipelines
Requires full-stack development skills
Longer development time
More code to maintain

Streamlit vs Shiny: Overview

Both frameworks enable data scientists to build interactive web applications without extensive web development knowledge

Shiny (R and Python):
R-first, now supports Python
Reactive programming model
Mature ecosystem in R
Enterprise features

Streamlit (Python):
Pure Python, script-based approach
Automatic reactivity
Rapid prototyping focus
Growing ecosystem

Largely most people say Streamlit shines in rapid development and ease of use, ideal for quick prototypes and data exploration tasks.

In comparison, Shiny is more of extensive framework for complex, feature-rich data apps, with PyShiny extending these capabilities to Python users.

Both platforms come with their unique strengths and limitations and deciding between the two depends on specific project requirements

Development Experience Comparison

Shiny:
Explicit reactive programming
Server-UI separation
More control over reactivity
R or Python (Shiny for Python)
Steeper learning curve but more flexibility

Streamlit:
Script runs top-to-bottom on each interaction
Implicit reactivity - no manual observers
Minimal boilerplate code
Python-first design
Faster initial development

High level comparison of Streamlit, RShiny, PyShiny

Criteria	Streamlit	RShiny	PyShiny
Language Ecosystem	Python-native	R-native	Python-native
Development Speed	Rapid Prototyping	Suited for both medium and long-term projects	Suited for both medium and long-term projects
Ease of Use	High (few lines of code)	Moderate to High (depends on customization)	Moderate to High (depends on customization)
Customization	Limited but improving	Advanced, pixel-level	Advanced, pixel-level
Community Support	Strong and growing (27k+ stars on GitHub)	Well-established and mature	Emerging, building upon RShiny community
Widgets	Increasing variety but less comprehensive	Wide range of pre-implemented widgets	Base implemented, supports all ipywidgets, more under development
Best For	Quick prototypes, data exploration	Ideal for medium-complex to feature-rich, robust dashboards	Ideal for medium-complex to feature-rich, robust dashboards in Python
Specialized Sectors	Fields requiring quick development cycles	Sectors needing varied complexity and strong reliability	Sectors needing varied complexity and strong reliability in Python
Examples	Proof-of-concept ML models, temporary dashboards	Scientific computing apps, enterprise-grade applications	Scientific computing apps, enterprise-grade applications in Python

Taken from https://www.appsilon.com/post/streamlit-or-shiny-for-life-scientists

posit.co has a good comparison (obviously pro-shiny) at https://shiny.posit.co/py/docs/comp-streamlit.html parts of it below.

Shiny and Streamlit differ in a few key ways

1. Shiny's reactive execution means that elements are minimally re-rendered.

2. You can build large Shiny applications without manually managing application state or caching data.

3. Shiny allows you to easily customize the look and feel of your application.

Shiny allows you to build much more performant and extensible applications than Streamlit. The patterns that you use to build a simple Shiny application are the same ones that you use to build a complex one, and you never need to change your mental model of how the application works. This design will let your application grow along with the scope of your problem, and you can have confidence that the framework has the tools that you need to handle almost any requirement.

Other Framework Options

Dash by plotly

Python framework for analytical web apps
Built on Flask, React, and Plotly
More control, steeper learning curve

gradio

Quick ML model demos and prototypes
Automatic UI generation from functions
Ideal for sharing models quickly

Panel

Works with Jupyter notebooks
Supports multiple plotting libraries
Flexible layout system

NiceGUI

Modern Python UI framework
Component-based architecture
Real-time updates with WebSockets

Core Streamlit Concepts

Script Re-runs: App reruns from top on interaction
Widgets: Interactive input elements
Caching: Optimize expensive operations
Session State: Persist data across reruns
Layout: Organize content with columns and containers
Theming: Customize appearance

Common Streamlit Widgets

Input: text_input, number_input, slider
Selection: selectbox, multiselect, radio
Buttons: button, checkbox, toggle
File: file_uploader, download_button
Media: image, video, audio
Data: dataframe, table, metric
Charts: line_chart, bar_chart, map

Building a Data Dashboard

Key components:

Data loading with caching

Interactive filters in sidebar

Multiple visualizations

KPI metrics at the top

Data export functionality

Responsive layout with columns

Multi-Page Applications

Create pages directory:

```
pages/
  1_Dashboard.py
  2_Analysis.py
  3_Settings.py
app.py
```

Streamlit automatically creates navigation

Each page is a separate Python file

Shared state via st.session_state

Best Practices for Streamlit Apps

Use caching extensively for performance
Provide clear instructions and labels
Handle errors gracefully with try-except
Show loading states with st.spinner
Validate user inputs before processing
Organize code with functions
Use session state for complex workflows

Getting Your App Online

Deployment Options

Streamlit Community Cloud:
Free hosting for public apps
Direct GitHub integration
Automatic deployments
Built-in secrets management

Google Cloud Run:
Containerized deployment
Pay-per-use pricing
Scalable infrastructure
More control and flexibility

Streamlit

Locally, in terminal

Git clone or zip download my streamlit-kmeans-418 —repo (not in your forked course repo!)

change into that directory and if you have python and streamlit installed on your machine you could simply run

```
streamlit run app.py
```

Or since there is also a Dockerfile (I don't have any python on this machine)

Preparing for Deployment

Required files:

- app.py - Your main application
- requirements.txt - Python dependencies
- README.md - Documentation
- .gitignore - Exclude sensitive files

Optional:

- .streamlit/config.toml - Theme configuration
- packages.txt - System dependencies

Create requirements.txt

List all dependencies:
streamlit
pandas
scikit-learn
requests
plotly

Best practice - Use specific versions
streamlit==1.28.0

Create a Dockerfile

```
FROM python:3.12-slim

WORKDIR /app

COPY . /app

RUN pip install --upgrade -r /app/requirements.txt

ENV PORT=8080
EXPOSE 8080

CMD ["sh", "-c", "streamlit run app.py --server.port=${PORT} --server.address=0.0.0.0 --
    server.enableCORS=false"]
```

Podman on Local Machine

Git clone or zip download my **streamlit-kmeans-418** (not in your forked course repo!)

Change your directory into the folder.

```
podman build -t my-app .
```

```
podman run --rm -p 8080:8080 my-app
```

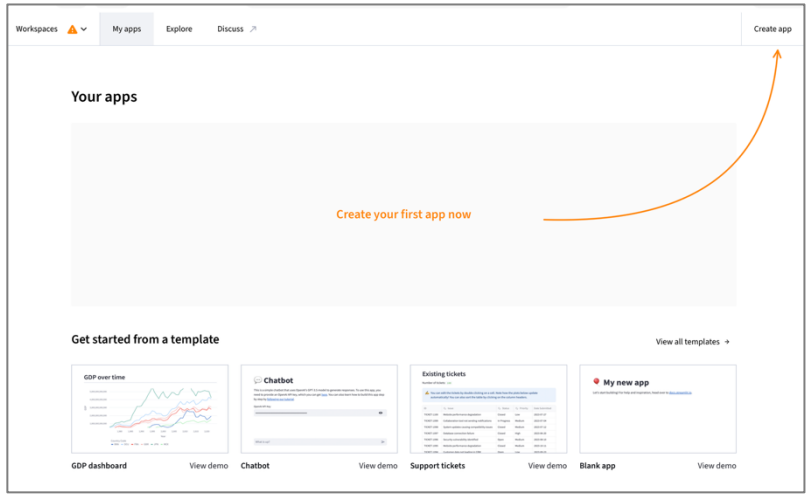
Now check http://localhost:8080

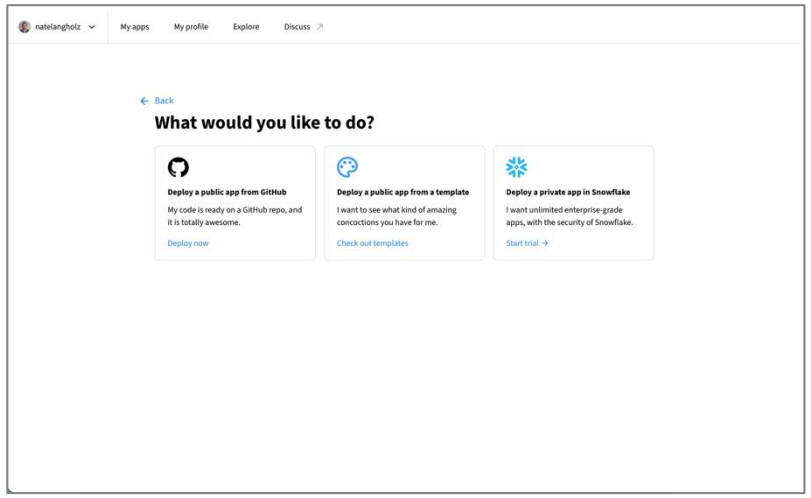
Streamlit Community Cloud Deployment

Steps:

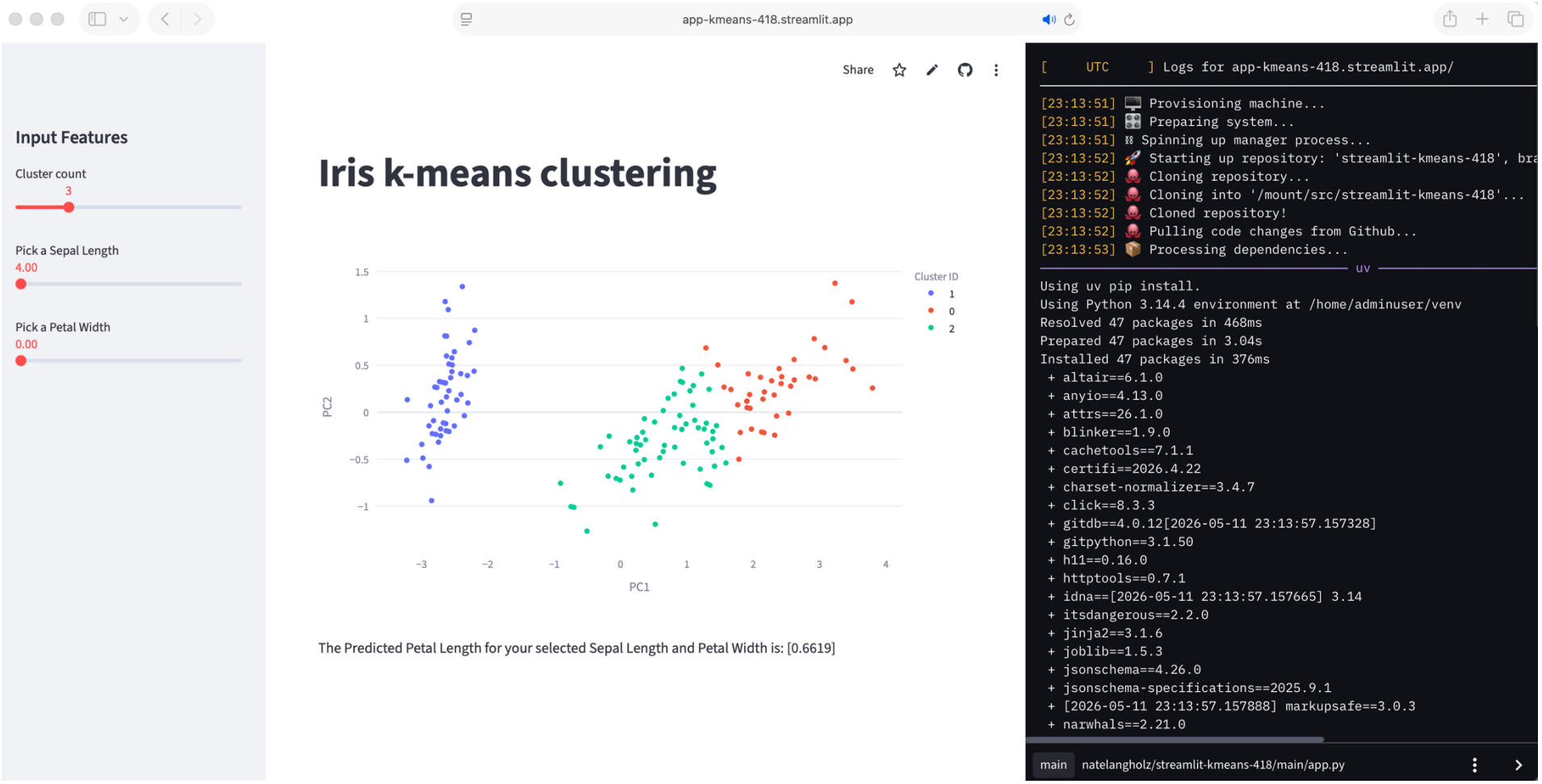
1. Push code to GitHub repository
2. Go to share.streamlit.io
3. Click ‘Create app'
4. Connect Github & Deploy public app
5. Select repository, branch, and main file
6. Click 'Deploy'

Your app will be live at:
your-app-name.streamlit.app





Streamlit Community Cloud Deployment



Managing Secrets in Streamlit Cloud

In app settings, add secrets:

```
API_KEY = 'your-api-key'
DATABASE_URL = 'your-db-url'
```

Access in code:

```
import streamlit as st
api_key = st.secrets['API_KEY']
```

Never commit secrets to Git!

Automatic Deployments

Streamlit Cloud watches your repository
Push to main branch triggers redeployment
View deployment logs in dashboard
Rollback to previous versions if needed

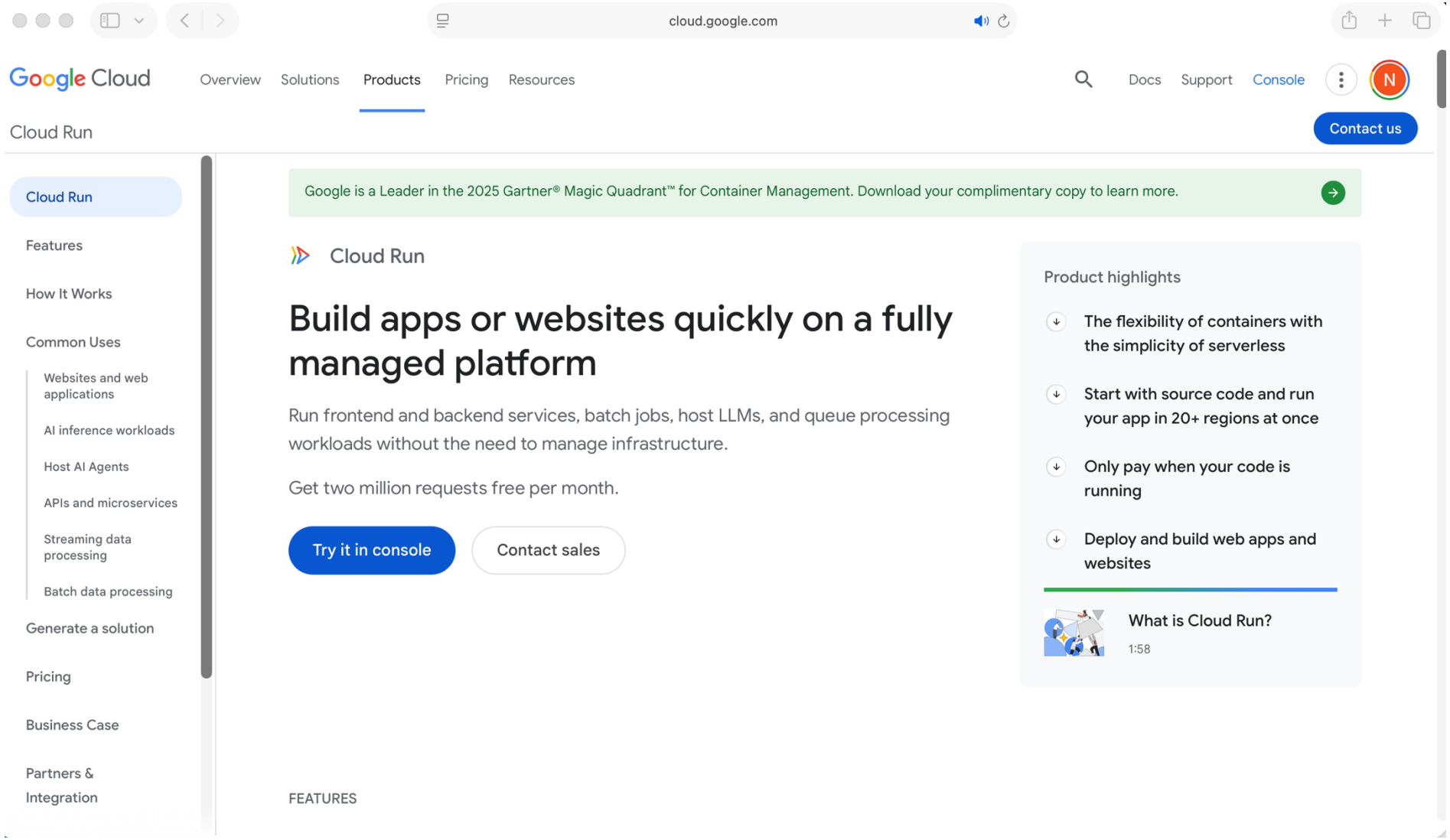
Development workflow:
Develop locally → Push to GitHub → Auto-deploy

Google Cloud Run Deployment

Why Cloud Run?

- Containerized applications
- Automatic scaling (0 to N instances)
- Pay only for actual usage
- Custom domains and SSL
- More control over environment
- Integration with GCP services

Google Cloud Run



console.cloud.google.com

Google Cloud | Gemini API | Search (/) for resources, docs, products, and more | Search

Cloud Run

- Overview
- Services
- Jobs
- Worker pools
- Domain mappings

Release Notes

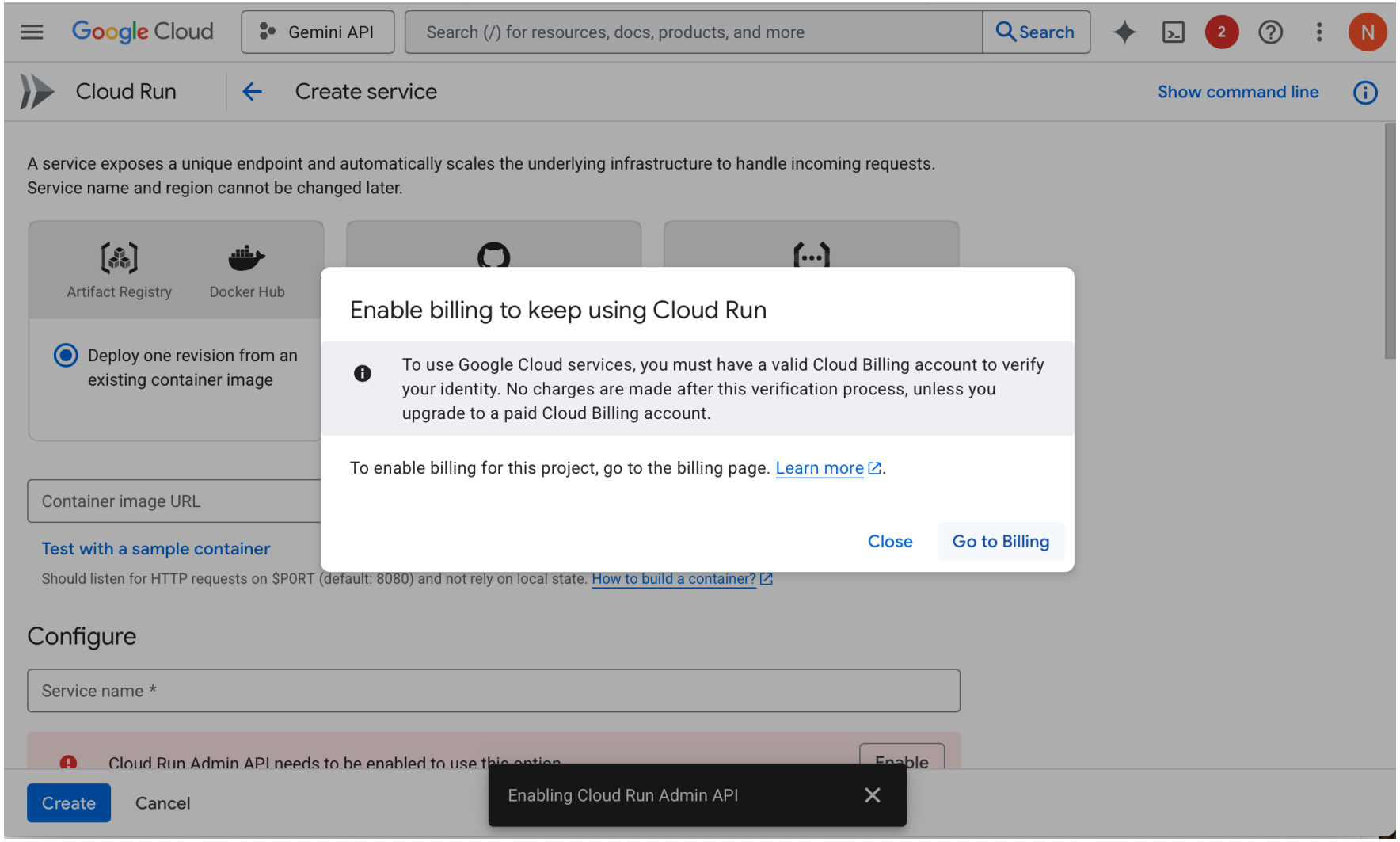
Services | Deploy container | Connect repo | Write a function | Refresh

A service exposes a unique endpoint and automatically scales the underlying infrastructure to handle incoming requests.
Deploy a container image, source code or a function to create a service.

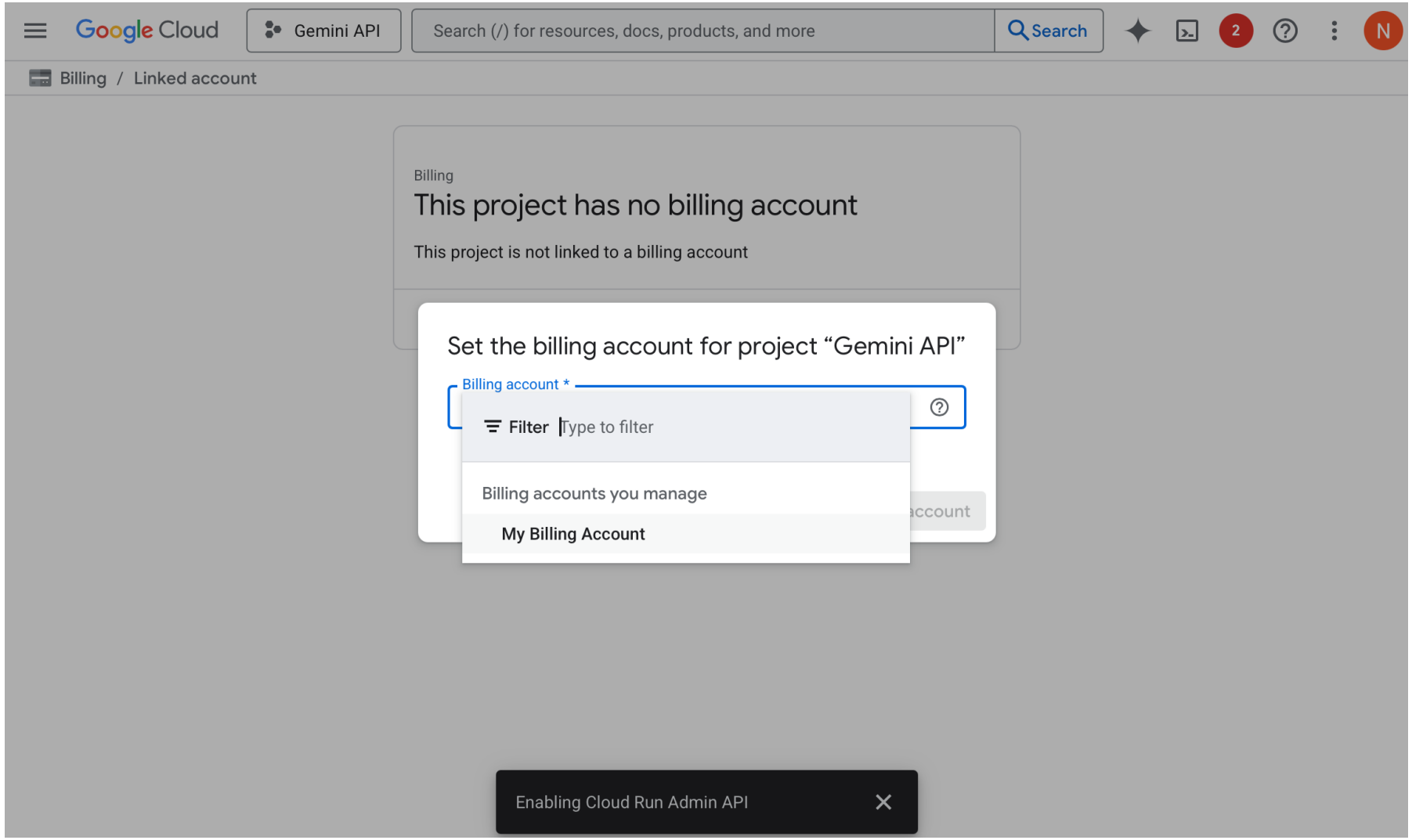
Filter | Filter services

Name	Deployment type	Req/sec	Region	Cost	Authentication	Ingress
plumber-app	Container	0.01	us-central1	–	Public access	All

You will need to enable billing to use



You will need to enable billing to use



Docker Hub

Container registries store and distribute built images.

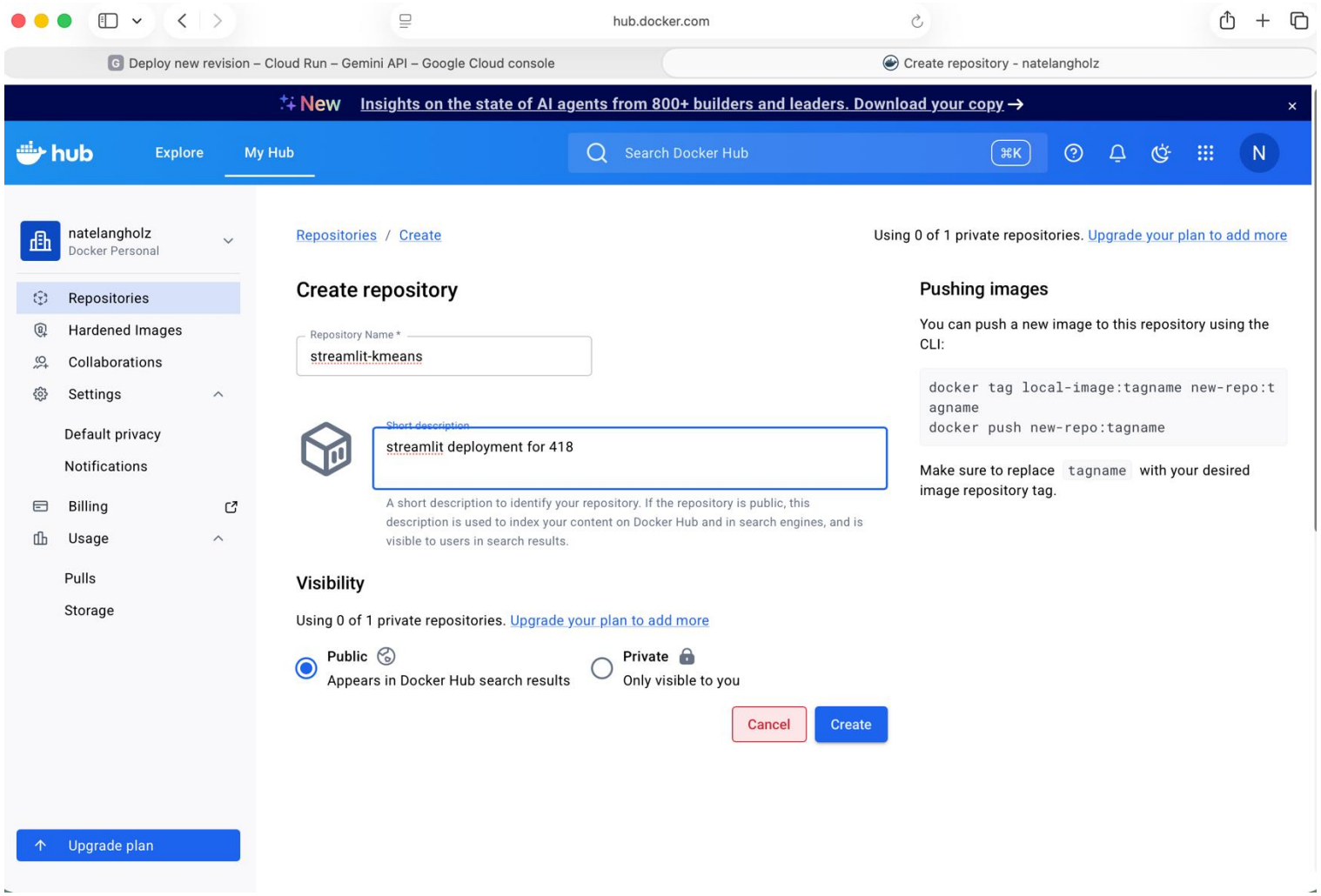
Docker Hub is the most common public registry
Other major registries:

- Google Artifact Registry
- Amazon ECR
- GitHub Container Registry
- Azure Container Registry
- Quay

Registries support:

- image versioning with tags (v1, v2)
- private and public images
- team access control
- CI/CD deployment workflows

Create an account and then repository where we will push our image.



Building and Pushing Container

Build Podman image:

```
podman build --platform linux/amd64 –t my-app .
```

Tag for Docker Hub:

```
podman tag my-app natelangholz/streamlit-kmeans
```

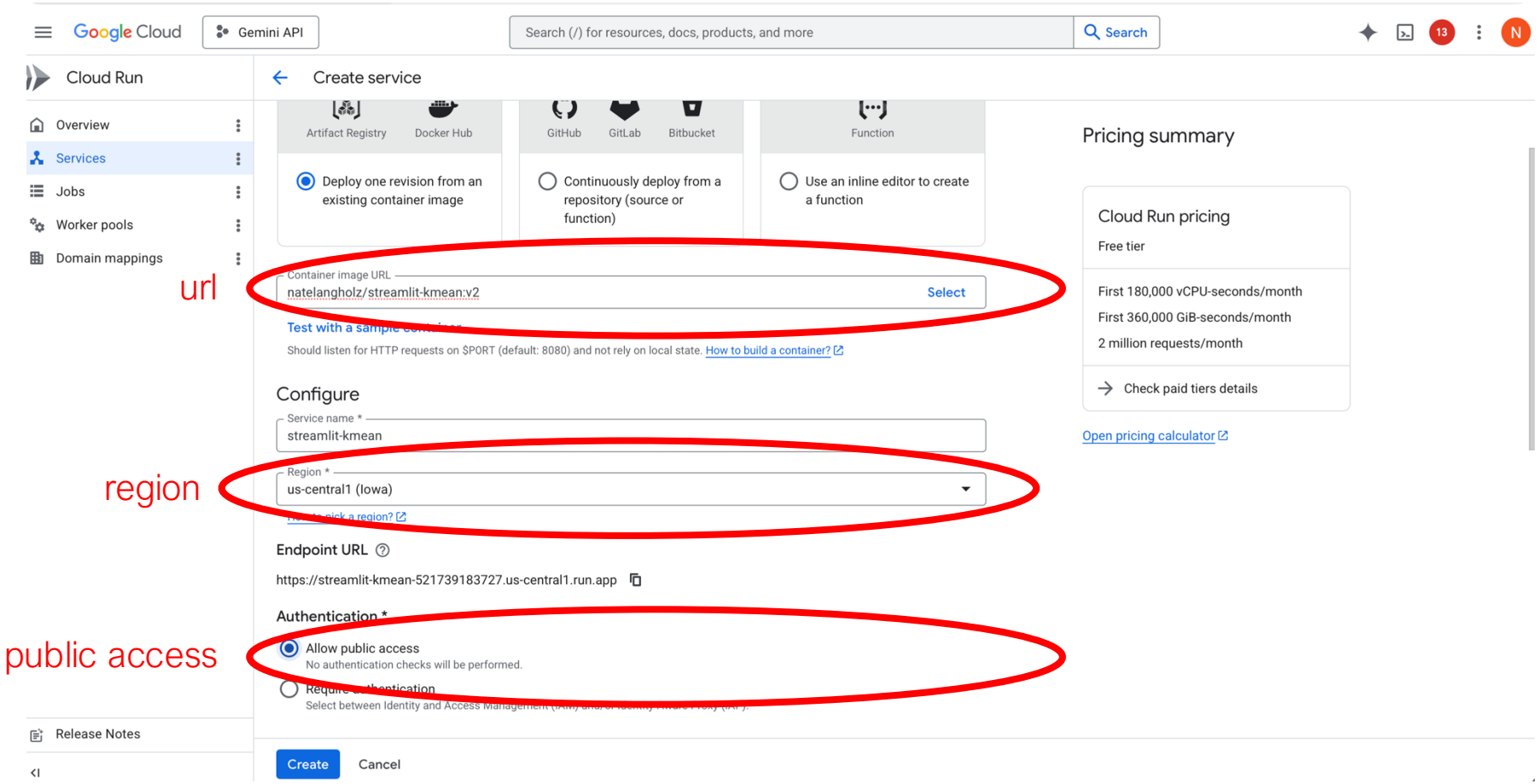
Login to Docker Hub:

```
podman login docker.io
```

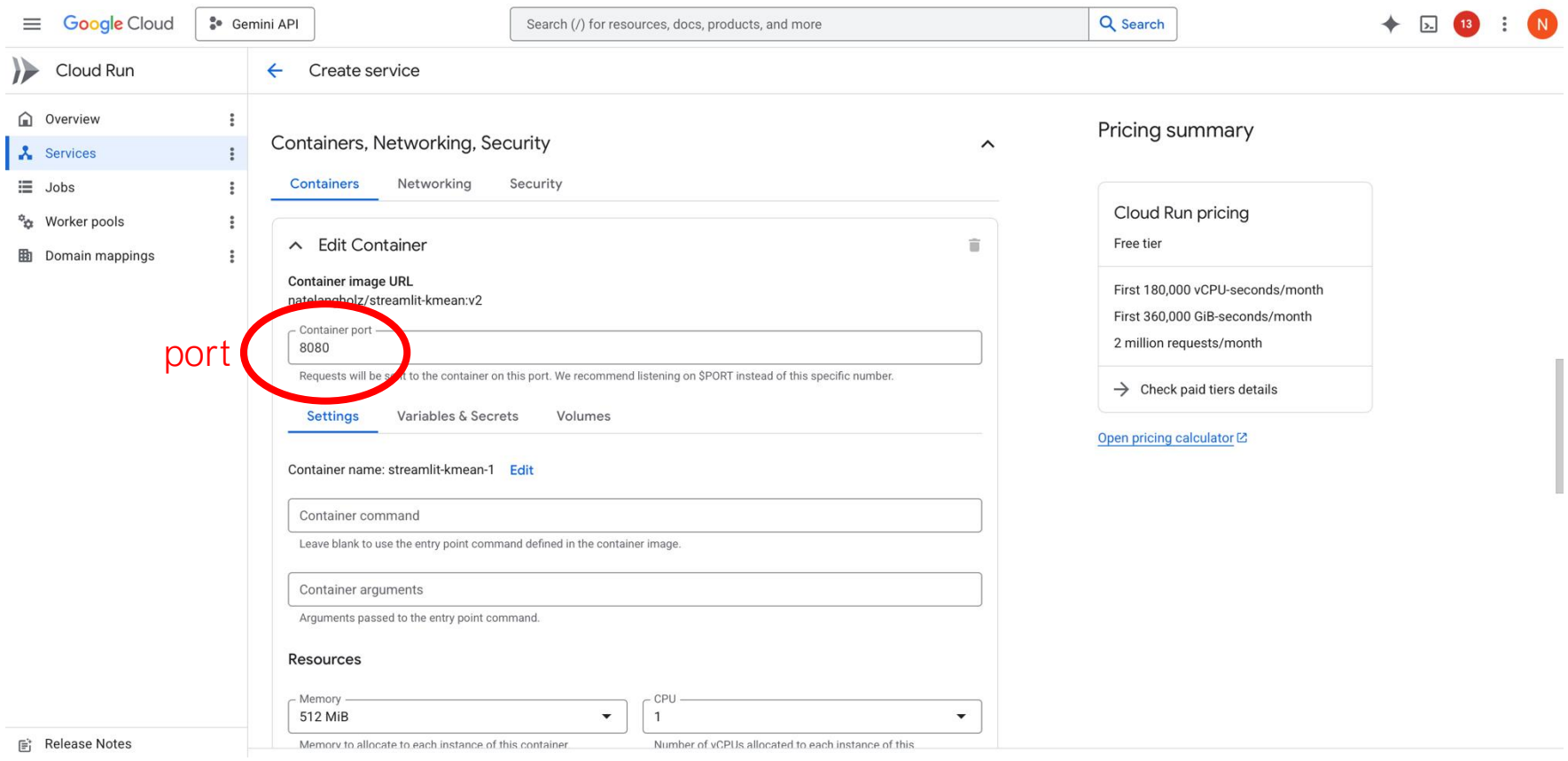
Push to registry:

```
podman push natelangholz/streamlit-kmeans
```

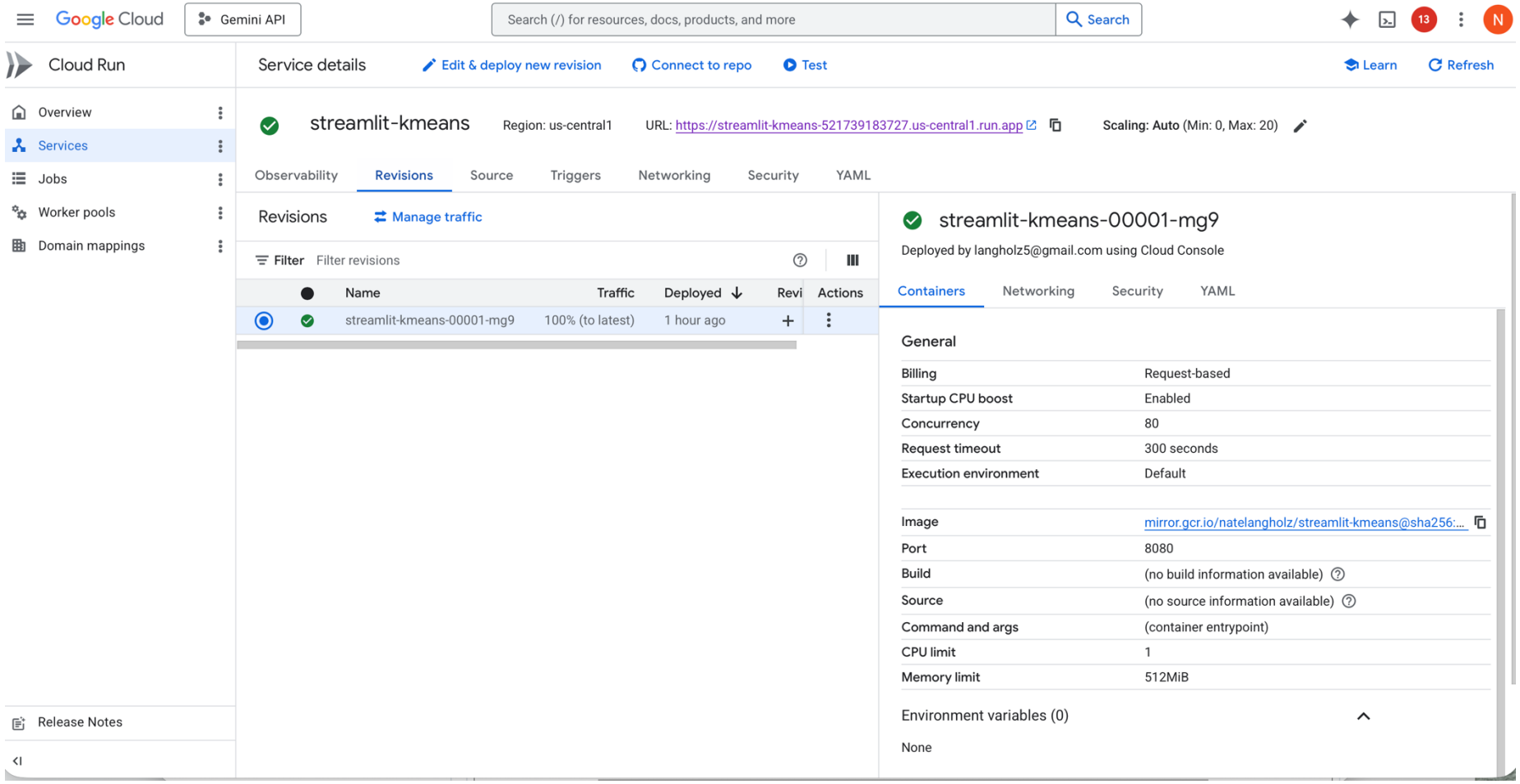
Enter your container image url, pick a region, allow public access



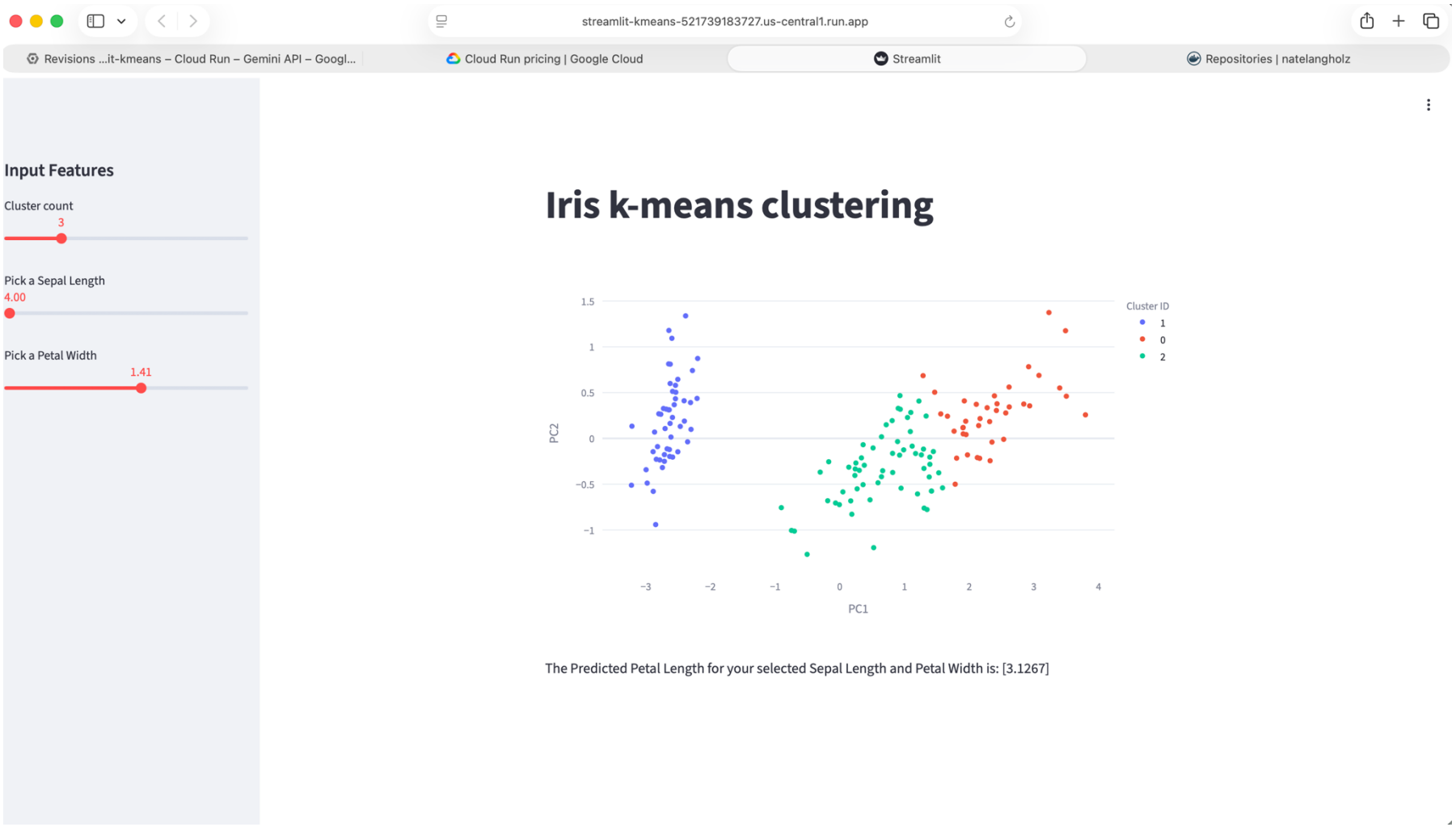
Confirm the port you expose in your Dockerfile matches container port.
Then you can create your service.



Your container will deploy and then you can access your app at the url



Success!

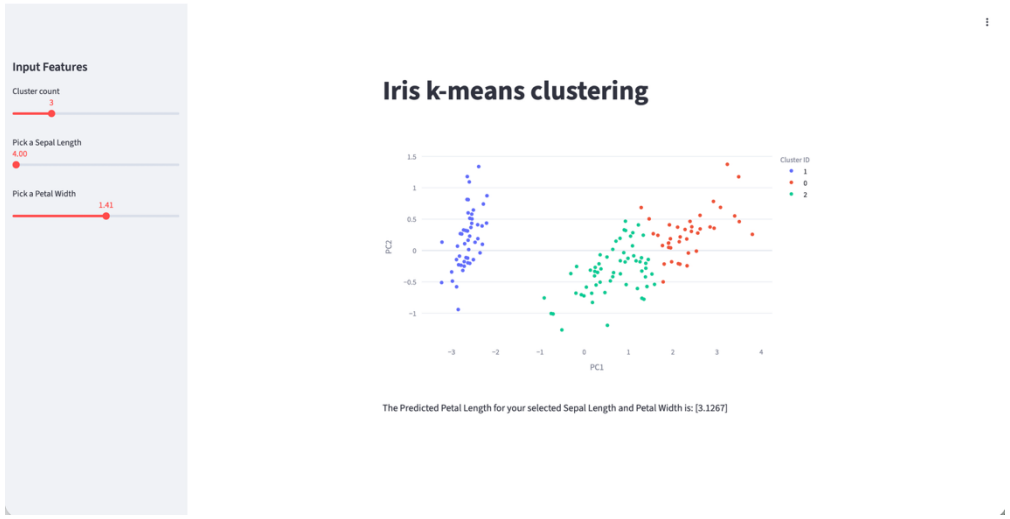

streamlit-kmeans-521739183727.us-central1.run.app
Revisions ...it-kmeans – Cloud Run – Gemini API – Googl...
Cloud Run pricing | Google Cloud
Streamlit
Repositories | natelangholz
Input Features
Cluster count
3
Pick a Sepal Length
4.00
Pick a Petal Width
1.41
Iris k-means clustering
Cluster ID
1
0
2
PC2
PC1
The Predicted Petal Length for your selected Sepal Length and Petal Width is: [3.1267]

Different Deployments

Streamlit

https://app-kmeans-418.streamlit.app

Cloud Run



https://streamlit-kmeans-521739183727.us-central1.run.app

Monitoring and Logging

Streamlit

Streamlit Community Cloud:
View logs in dashboard
Monitor app performance
Track user sessions

Cloud Run

Google Cloud Run:
Cloud Logging for application logs
Cloud Monitoring for metrics
Set up alerts for errors
Analyze traffic patterns

Cost Considerations

Streamlit

Streamlit Community Cloud:
Free for public apps
Limited resources per app
Shared infrastructure

Cloud Run

Google Cloud Run:
Pay per request and compute time
Free tier: 2M requests/month
Scales to zero when not in use
Predictable pricing model

Deployment Best Practices

Test locally before deploying
Use version control (Git)
Document deployment process
Implement error handling
Monitor application performance
Set up automated testing
Keep dependencies updated
Plan for scaling needs

Taking Your Apps Further

Authentication and Access Control

Simple password protection:

```
import streamlit_authenticator as stauth
```

OAuth integration:

Google, GitHub, Microsoft authentication

Role-based access:

Different views for different users

Admin vs regular user permissions

Database Integration

SQLite for simple apps:

```
import sqlite3
conn = sqlite3.connect('data.db')
```

PostgreSQL for production:

```
import psycopg2
conn = psycopg2.connect(DATABASE_URL)
```

Cache connections:

```
@st.cache_resource
def get_connection():
    return create_connection()
```

Real-time Data Updates

Auto-refresh pattern:

```
placeholder = st.empty()
while True:
    data = fetch_latest_data()
    with placeholder.container():
        display_data(data)
    time.sleep(60)
```

WebSocket connections for live data

Polling vs push-based updates

Performance Optimization

Caching strategies:

- Cache data loading and transformations
- Cache model predictions
- Cache API responses

Lazy loading:

- Load data only when needed
- Paginate large datasets

Optimize computations:

- Vectorize operations with NumPy/Pandas
- Use efficient data structures

Testing Streamlit Apps

Unit tests for functions:

```
import pytest
def test_data_processing():
    result = process_data(sample_data)
    assert result is not None
```

Integration tests:

Test app components together

Mock external dependencies

Manual testing checklist

Error Handling and Debugging

Graceful error handling:

```
try:
    result = risky_operation()
except SpecificError as e:
    st.error(f'Operation failed: {e}')
    st.stop()
```

Debugging tips:

- Use `st.write()` to inspect variables
- Check browser console for errors
- Review Streamlit logs

Security Considerations

Never expose secrets in code
Validate all user inputs
Sanitize data before display
Use HTTPS in production
Implement rate limiting for APIs
Regularly update dependencies
Follow OWASP guidelines

Scaling Considerations

Horizontal scaling:
Multiple instances behind load balancer
Stateless application design

Vertical scaling:
Increase memory and CPU
Optimize resource usage

Caching layers:
Redis for shared cache
CDN for static assets

CI/CD for Streamlit Apps

GitHub Actions workflow:
Run tests on push
Build Docker image
Deploy to Cloud Run

Automated testing:
Lint code with flake8
Type checking with mypy
Security scanning

Continuous deployment pipeline

Documentation Best Practices

README.md should include:
Project description and purpose
Installation instructions
Usage examples
Deployment guide
Contributing guidelines

In-app documentation:
Clear labels and tooltips
Help sections with st.expander
Example inputs

Common Pitfalls to Avoid

Not using caching (slow performance)
Ignoring error handling (poor UX)
Hardcoding secrets (security risk)
Not validating inputs (bugs and security)
Overcomplicating the UI (confusion)
Not testing before deployment
Ignoring mobile responsiveness